

Description of data structures used:

We used two custom linked lists (`instrumentList`) that contain orders (`instrumentNode`), one for buy and one for sell resting orders. This ensures that the buy orders and sell orders are accessed independently of each other. The buy list is sorted in descending order while the sell list is sorted in ascending order. This is done using an `isAsc` field whose value will be set as false for `buyList` and true for `sellList` when the `instrumentLists` are initialized.

When inserting a node into the sell list, we iterate through the list and check if the price of the order to be inserted is $<$ price of the current `instrumentNode`. This ensures that the list is sorted by price following the `isAsc` field and if there are orders of the same price, the orders are sorted such that orders added to the list at a later time will be at the back. This is similar for the buy list.

Each node will also contain an `instrument` field that is required for the matching operations.

Each of these lists contains a dummy node that will always be at the end of the list. This is ensured by setting the price of the dummy node in the sell list at `UINT32_MAX` and the dummy node in the buy list at 0. As I iterate through the list, I check if it is a dummy node to know if I have reached the end of the list. A dummy node is required as I am doing a hand-over-hand locking where I acquire the lock of the next node before moving on to the next node to prevent data races. I will do `lock_curr = std::move(lock_next);` to make use of the same `lock_curr` variable and unlock the mutex that was previously in `lock_curr`.

Synchronisation primitives used:

To ensure thread-safe access to shared data structures, a combination of mutexes and atomics are used. Each buy and sell list is protected by its own 'head' mutex called `head` to prevent data races whenever the same list is being accessed. This is required as the head of the list can be modified during order insertion when an order is added to the list.

Each node also stores a mutex to ensure exclusive access by one thread for each node at any point in time. This means that our resting orders which have an `exe_id` field can be incremented safely whenever it matches with an order. After it acquires the lock inside the first node and starts iterating through the list, it will unlock the head mutex of the list to improve concurrency unless a node is to be inserted to the start of the list.

Before iterating to the next node, it will acquire the lock in variable `lock_next` and move it to `lock_curr` and unlock the mutex inside the current node. This prevents data races that arise from accessing the next node of the current node.

For each `addToList` operation, on top of `lock_curr`, we also use an additional lock `lock_prev`. This ensures that we can safely add a new node as accesses to the node before and after the new node's position are synchronised.

Inside the engine struct, we have two atomic integer variables called `sellMatchingCount` and `buyMatchingCount` which helps us achieve phase level concurrency. It has to be atomic since these variables can be accessed by multiple threads to prevent data races.

Level of concurrency:

We achieved phase level concurrency by allowing multiple buy operations or multiple sell operations, where orders of different instruments can execute concurrently as they do not conflict with each other. Cancellations can be done regardless of whether it is currently executing buy or sell orders.

This is managed using atomic counters (`buyMatchingCount` and `sellMatchingCount`) with acquire-release. For consecutive buy orders, each buy order will increment its `buyMatchingCount` by 1. Incoming sell orders will busy wait while `buyMatchingCount.load(std::memory_order_acquire) != 0`. However, after the first buy order is done with its operations, it will busy wait while `buyMatchingCount.load() != 1` to ensure that the other buy orders will finish executing first. The moment the first buy order is the last order left in the consecutive buy orders, it will do `buyMatchingCount.fetch_sub(1, std::memory_order_release)` to decrement its counter to 0 to allow sell orders to proceed. This is similar for multiple sell orders, ensuring buy and sell phases do not overlap. Since I am using acquire release, there will be a synchronizes with relationship between a buy phase and a sell phase which establishes a happens before relationship between them.

For cancellation of orders we set its count to be 0. Resting orders whose count becomes 0 will be kept in the list instead of being deleted. This prevents the synchronisation problems that might arise from the deletion of `instrumentNodes`. When we cancel an order, we reject a cancellation if the count of that order id is 0 or if both `buyList` and `sellList` do not contain a resting order of that id, else it will be accepted.

When adding orders of new instruments, it would just be an `addToList` operation that can be done concurrently without blocking other `addToList`, `match` and `cancel`

operations. This is because orders of different instruments but the same side are added to the same list.

When adding orders for new instruments, our implementation ensures that the operation can be performed concurrently without blocking other `addToList`, `match` or `cancel` operations. Multiple threads are able to add orders for different instruments concurrently, as long as they are on the same side (buy or sell). This is achieved by utilizing the fact that orders for different instruments but the same side (buy or sell) are added to the same list.

Our implementation uses fine-grained locking at the node level to ensure that modifications to the list (`addToList`) do not block other operations. For example, when inserting a new order, this implementation locks only the relevant portion of the list (using hand-over-hand locking) rather than locking the entire list. This minimizes contention and allows other threads to continue their operation. Thus, the system achieves concurrency, even when adding new instruments or processing a large number of orders.

Testing methodology:

We use the `gen.py` file to generate test cases for single and multi-threading testing. `gen.py` basically takes in the number of threads and test cases as input. For each thread, `gen.py` will also keep track of which thread input which order to maintain semantics of the test case requirement. When generating the test cases, the order number is atomically incremented to prevent any two orders from having the same order number, while for the other components, they are auto-generated. Collisions between the orders of the same instrument are tested by reducing the permutation size of the possible instrument name to 26, hence increasing the likelihood of an active order fulfilled by the resting orders. By varying the number of threads for the `gen.py` generator, it ensures that we can test if the engine is thread-safe and data races are absent when there are concurrent executions.